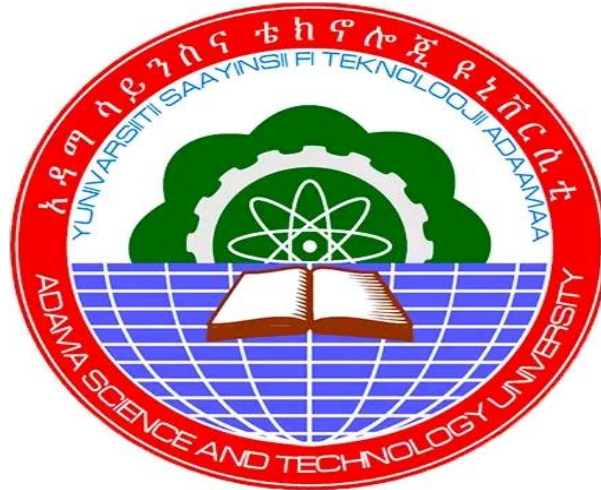




ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING

SOFTWARE ENGINEERING PROGRAM

Computer Architecture and Organization

Individual Assignment I

Course code: CSEg3202

Name: Aman Abdela

ID: ugr/30134/15

Section: 3

Submitted to: Mr. Kassahun Tamir

Submission date: December 26, 2024

1.1 General Purpose Registers

ARM architecture typically has 16 general-purpose registers named R0 to R15. Each register is 32 bits long in the ARM32 architecture and 64 bits long in the ARM64 architecture. These registers are used for various operations, including arithmetic calculations, data storage and function calls.

1.2 Special Purpose Registers

In addition to the general-purpose registers, ARM includes several special-purpose registers, each serving a specific function:

- **R15 (PC - Program Counter):** The Program Counter holds the address of the next instruction to be executed. It is automatically incremented after each instruction fetch.
- **R14 (LR - Link Register):** The Link Register stores the return address when a function call is made. This allows the program to return to the correct location after executing a function.
- **R13 (SP - Stack Pointer):** The Stack Pointer indicates the current top of the stack, which is used for managing function parameters, local variables and return addresses.
- **CPSR (Current Program Status Register):** The CPSR contains flags that reflect the status of the processor, including condition codes (negative, zero, carry, overflow) interrupt disable bits and the current processor mode (user, supervisor, etc.).

2. ARM Instruction Set

2.1 Instruction Format

ARM instructions are encoded in various formats, allowing for flexibility and efficiency in instruction execution. The primary instruction formats include:

Data Processing Instructions:

- **Format:** OPCODE S Rn, Rm and Rm2
- **Description:** Specifies the operation (e.g., ADD, SUB) and the operands.
- **Example:** ADD R0, R1, R2 (adds the values in R1 and R2, storing the result in R0).

• Load/Store Instructions:

- **Format:** LDR/STR Rn, [Rm, #offset]
- **Description:** Used to load data from memory into a register or store data from a register into memory. **Example:** LDR R0, [R1, #4] (loads data from the address in R1 plus an offset of 4 into R0).

• Branch Instructions:

- **Format:** B label
- **Description:** Changes the program flow to the specified label.
- **Example:** B loop (branches to the instruction labeled 'loop').

• Control Instructions:

- **Format:** BL label

- **Description:** Branch with link, saves the return address in LR.
- **Example:** BL function (branches to a function and saves the return address).

2.2 Instruction Types

The ARM instruction set includes various types of instructions designed for different functionalities:

- **Data Processing:**
 - **Example:** SUB R3, R4, R5 (subtracts R5 from R4 and stores the result in R3).
- **Load/Store:**
 - **Example:** STR R2, [R1] (stores the value of R2 into the address pointed to by R1).
- **Control Flow:**
 - **Example:** BNE label (branches to a label if the last comparison was not equal).
- **Multiplication:**
 - **Example:** MUL R1, R2, R3 (multiplies R2 and R3, storing the result in R1).

2.3 Data Types

ARM supports multiple data types which are crucial for programming:

- **Integer Types:**
 - **8-bit** (byte), **16-bit** (halfword), **32-bit** (word) and **64-bit** (double word).
- **Floating-point Types:**
 - **Single precision** (32-bit) and **double precision** (64-bit), used for handling real numbers.
- **Vector Types:**
 - **SIMD (Single Instruction, Multiple Data)** types, which allow for parallel processing of multiple data points.

2.4 Addressing Modes

ARM architecture offers various addressing modes, enabling flexible access to data:

- **Immediate Addressing:**
 - ✓ **Description:** Directly specifies a constant value.
 - ✓ **Example:** MOV R0, #5 (loads the value 5 into R0).
- **Register Addressing:**
 - ✓ **Description:** Uses a register to specify the address.
 - ✓ **Example:** LDR R1, [R2] (loads data from the address contained in R2).
- **Base Offset Addressing:**
 - ✓ **Description:** Combines a base address in a register with an offset.
 - ✓ **Example:** LDR R3, [R4, #8] (loads data from the address in R4 plus an offset of 8).
- **PC-relative Addressing:**
 - ✓ **Description:** Uses the program counter plus an offset to determine the target address.
 - ✓ **Example:** B label (the target address is calculated based on the current PC).

- **Register Indirect Addressing:**

- ✓ **Description:** Uses a register to hold the address of data.

- ✓ **Example:** STR R0, [R1, R2] (stores the value of R0 into the address pointed to by R1 plus the offset in R2).

3. Cache Organization

ARM architecture incorporates various cache features to enhance processing speed and efficiency.

3.1 Cache Levels

ARM typically employs multi-level cache systems, including:

- **L1 Cache:**

- **Description:** The fastest and closest cache to the CPU, generally split into instruction and data caches (I-cache and D-cache).

- **Example:** 32 KB instruction cache and 32 KB data cache, often 4-way set associative.

- **L2 Cache:**

- **Description:** Larger than L1 cache, providing a second level of caching.

- **Example:** 256 KB unified cache, which may be shared among multiple cores in a multicore processor.

- **L3 Cache** (if applicable):

- **Description:** Further increases cache capacity and reduces latency for multi-core processors.

- **Example:** 1 MB or more typically shared among all cores.

3.2 Cache Line Size

The cache line size, which is the smallest unit of data that can be transferred between cache and memory, typically ranges from 32 to 64 bytes.

3.3 Cache Associativity

ARM caches can be configured in different associative ways:

- **Direct-mapped:** Each block of main memory maps to exactly one cache line.

- **Set-associative:** Each block can map to several cache lines, allowing for greater flexibility.

- **Fully associative:** Any block can go into any cache line, minimizing collisions.

3.4 Write Policies

ARM caches support different write policies to manage how data is written:

- **Write-back:** Data is written to the cache first and only written to memory when it is evicted from the cache.

- **Write-through:** Data is written to both the cache and the memory simultaneously, ensuring memory always holds the most recent data.

3.5 Cache Coherency

In multicore processors, maintaining cache coherency is essential to prevent inconsistencies.

ARM employs protocols like MESI (Modified, Exclusive, Shared, Invalid) to ensure that all

cores see the same data at all times.

3.6 Example of Cache Features

For instance, an ARM Cortex-A processor may implement:

- **L1 Cache:** 32 KB I-cache and 32 KB D-cache, 4-way set associative.
- **L2 Cache:** 256 KB unified cache, 8-way set associative, shared among cores.
- **L3 Cache:** 1 MB shared cache, utilized in higher-end models.

Conclusion:

The ARM architecture is a robust and efficient platform for modern computing characterized by its well-organized register structure, diverse instruction set and sophisticated cache organization. Its features make it particularly suitable for embedded systems, mobile devices and increasingly for high-performance computing applications. The balance between performance, power efficiency and flexibility has established ARM as a leader in the processor market. Through its ongoing evolution, ARM continues to adapt to the demands of new technologies, maintaining its relevance in a rapidly changing landscape.

